

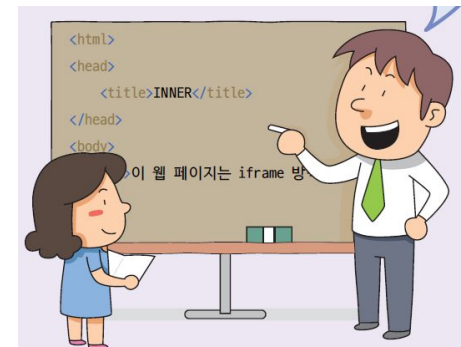
CHAPTER 15.

HTML5의 웹스토리지, 파일API, AJAX, 웹소켓



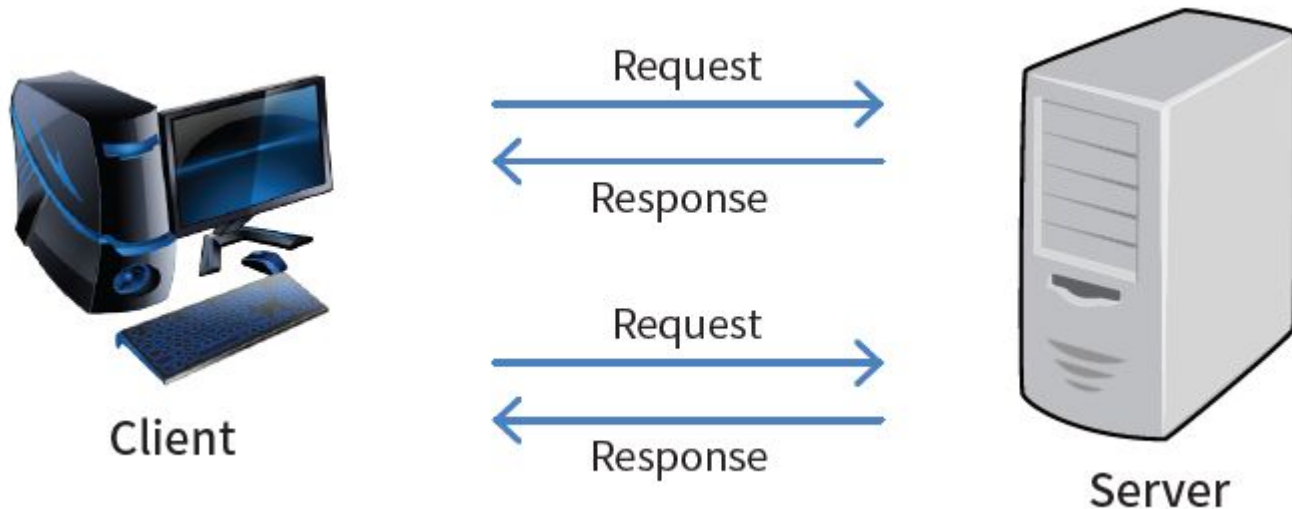
15장의 목표

- 쿠키가 왜 필요한지 이해할 수 있나요?
- 웹스토리를 사용할 수 있나요?
- 파일 **API**를 이용하여서 파일을 업로드할 수 있나요?
- **AJAX**가 무엇인지 설명할 수 있나요?



HTTP 프로토콜과 쿠키

- HTTP(HyperText Transfer Protocol)는 1996년 HTTP/1으로 등장한 이후로 약간의 변경이 있었지만, 근본적으로 “무상태” 프로토콜(stateless protocol)이다. “무상태”이라고 하면 서버가 두 요청(request) 간에 상태를 유지하지 않음을 의미한다.



상태 프로토콜의 이해

- 우리가 백화점에서 TV를 구매한다고 하자. 다음과 같은 대화가 오갈 수 있다. 백화점 직원은 상태를 기억하고 있다.

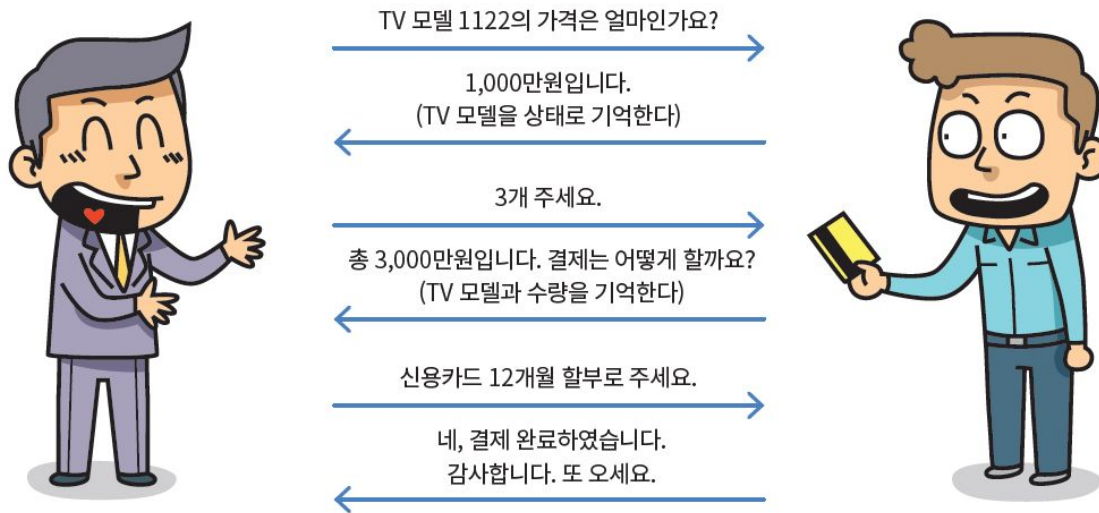


그림 15-1 상태 프로토콜의 예

무상태 프로토콜의 이해

- 하지만 어떤 직원은 기억력에 문제가 있어 직전의 대화를 기억하지 못한다고 하자. 다음과 같이 혼란이 발생할 것이다.

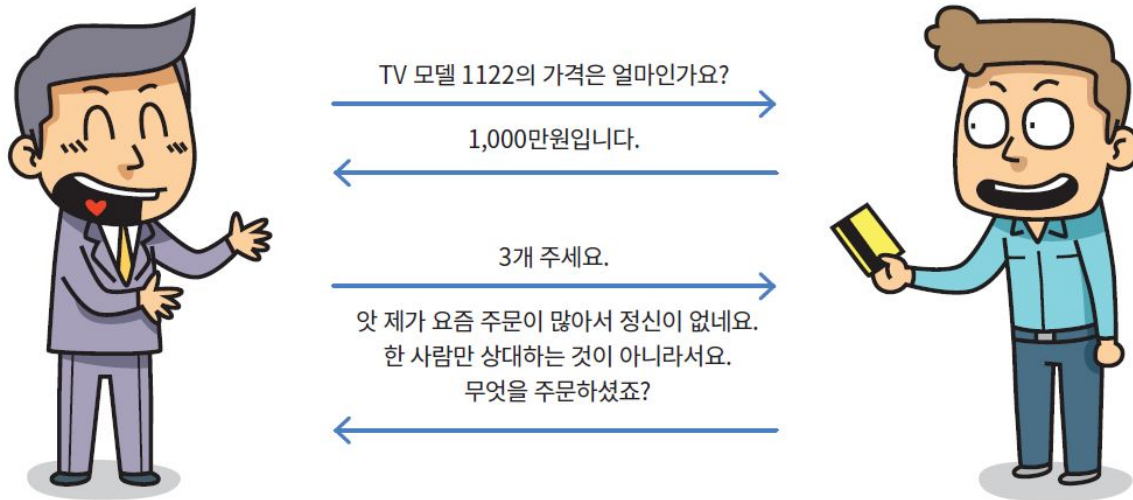
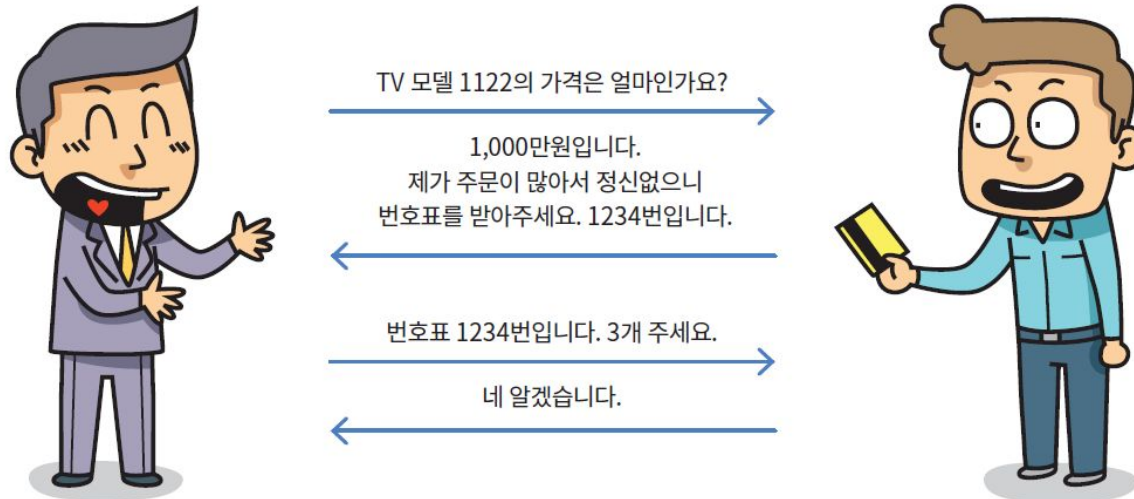


그림 15-2 무상태 프로토콜의 예

쿠키(cookie)란 무엇인가?

- 애플리케이션은 클라이언트 컴퓨터에 무엇인가를 저장할 필요가 있다. 예를 들어서 장바구니에 담긴 물건이나 자신이 방문한 횟수와 같은 정보들을 저장할 필요가 있는 것이다. 이전에는 이것을 쿠키(cookie)로 해결하였다.



쿠키의 특징

- 웹 서버에서 사용자의 웹 브라우저로 보내지는 데이터이다.
- 쿠키는 단순한 텍스트 데이터이다. 이진 데이터가 아니다.
- 쿠키는 브라우저에 의해 사용자의 컴퓨터(디스크)에 저장된다.
- 웹사이트는 자체 쿠키만 읽을 수 있다. 다른 웹사이트의 쿠키를 읽을 수 없다. 이 보안은 브라우저에 의해 보장된다.
- 쿠키는 다른 브라우저 간에 공유되지 않는다. 즉, 동일한 도메인이더라도 한 브라우저는 다른 브라우저에 저장된 쿠키를 읽을 수 없다.
- HTTP 프로토콜에 따라 쿠키의 크기는 **4KB**를 초과할 수 없다.

크롬에서 쿠키 보는 방법

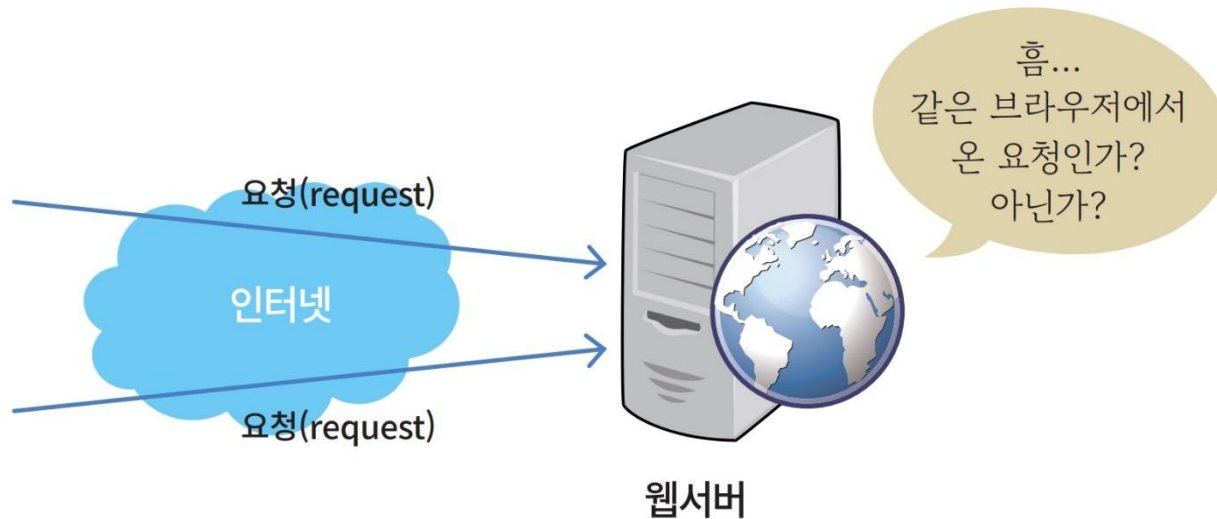
The screenshot shows the Google Chrome browser interface. On the left, the 'Storage' menu is open, and the 'Cookie' option is highlighted with a red box. The main content area displays a table of cookies. The table has the following columns: 이름 (Name), 값 (Value), D... (Domain), Pa... (Path), Ex... (Expires), 크... (Size), Ht... (HTTPOnly), Se... (Secure), Sa... (SameSite), Pa... (Partition), and P... (Priority). The table lists various cookies, including _Secure-3PSID, _Secure-3PSID, _Secure-1PSID, SID, _Secure-1PSI..., SIDCC, _Secure-3PAP..., SAPISID, APISID, SSID, _Secure-1PAP..., HSID, UULE, 1P_JAR, OGPC, and SFARCH SAM....

이름	값	D...	Pa...	Ex...	크...	Ht...	Se...	Sa...	Pa...	P...
__Secure-3PSI...	AJi4QfGez0AoDjLAeyZm...	.g...	/	20...	91	✓	✓	N...		Hi...
__Secure-3PSID	LAg7EhEx5FUp38grppbJf...	.g...	/	20...	85	✓	✓	N...		Hi...
__Secure-1PSID	LAg7EhEx5FUp38grppbJf...	.g...	/	20...	85	✓	✓		✓	Hi...
SID	LAg7EhEx5FUp38grppbJf...	.g...	/	20...	74					Hi...
__Secure-1PSI...	AJi4QfHQ81aoQNR8OFv...	.g...	/	20...	91	✓	✓			Hi...
SIDCC	AJi4QfEerwronXthF-CRZ...	.g...	/	20...	80					Hi...
__Secure-3PAP...	hTYAW_RIR7xs4nt_/Akg8...	.g...	/	20...	51		✓	N...		Hi...
SAPISID	hTYAW_RIR7xs4nt_/Akg8...	.g...	/	20...	41		✓			Hi...
APISID	F4r5UCYf8wpk9TCh/AAr...	.g...	/	20...	40					Hi...
SSID	A6uTnp9TpX2hIAel2	.g...	/	20...	21	✓	✓			Hi...
__Secure-1PAP...	hTYAW_RIR7xs4nt_/Akg8...	.g...	/	20...	51		✓		✓	Hi...
HSID	A2i6F873ZY0v8WWFR	.g...	/	20...	21	✓				Hi...
UULE	a+cm9sZTogMQpwcw9k...	w...	/	20...	210		✓			M...
1P_JAR	2022-07-01-07	.g...	/	20...	19		✓	N...		M...
OGPC	19025836-2:	.g...	/	20...	15					M...
SFARCH SAM...	CnOI4nIR	.n...	/	20...	23			St...		M...

이것이 쿠키이다.

쿠키가 필요한 이유

- 인증 (세션 관리)
- 사용자 추적
- 개인화 (테마, 언어 선택 등)

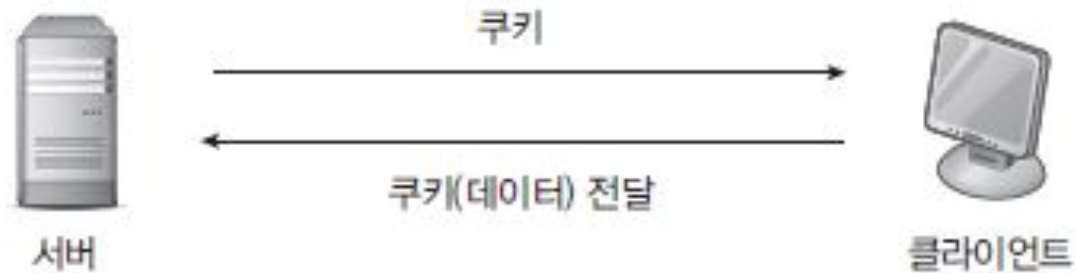


쿠키의 동작 과정

- ① 브라우저는 웹 서버에 **HTTP** 요청을 보낸다.
- ② 웹 서버에 요청이 도착하면 서버 측 코드가 사용자 이름과 암호를 확인한다. 로그인이 성공하면 서버는 일종의 세션 아이디 역할을 하는 쿠키를 포함한 **HTML** 페이지를 보낸다.
- ③ 이 쿠키는 **Set-Cookie** 헤더를 사용하는 **HTTP** 응답의 일부로 전송된다.
- ④ 브라우저는 이 쿠키를 디스크에 저장한다.
- ⑤ 이제 사용자가 웹 서버의 다른 페이지로 이동하면 브라우저가 **HTTP** 요청의 일부로 이 쿠키를 자동으로 보낸다.
- ⑥ 웹 서버는 이 쿠키를 읽고 유효성을 식별한다.



쿠키 COOKIE



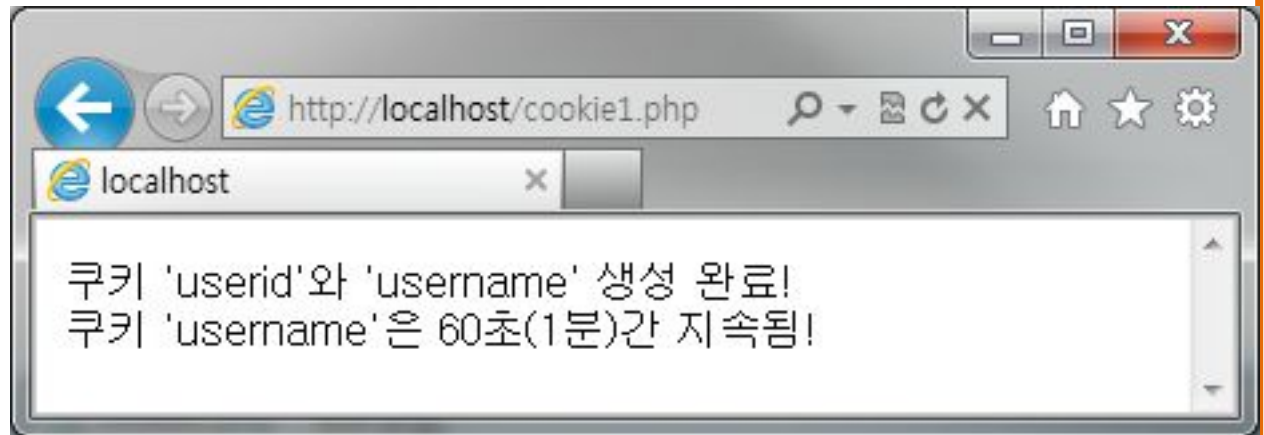
	서버	클라이언트
쿠키 저장 위치		✓
쿠키(정보)		✓
데이터 가공		✓

[그림] 쿠키의 개념과 관련 정보의 처리 장소

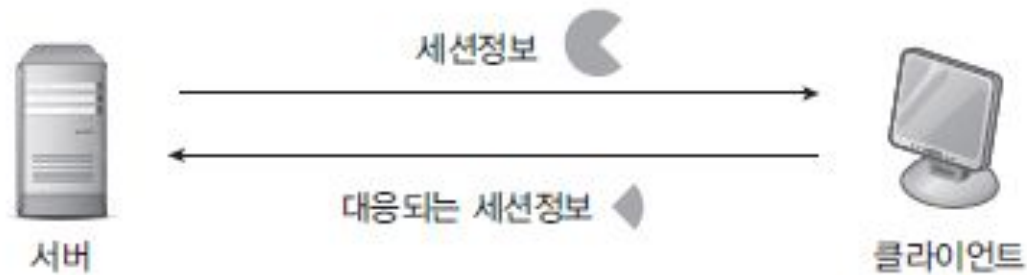
예제 setcookie()함수를 이용한 쿠키 생성

cookie1.php

```
01 <?
02 $a=setcookie("userid", "kdhong");
03 $b=setcookie("username","홍길동", time()+60);
04
05 if($a and $b)
06 {
07     echo "쿠키 'userid'와 'username' 생성 완료!<br>";
08     echo "쿠키 'username'은 60초(1분)간 지속됨!";
09 }
10 ?>
```



세션 SESSION



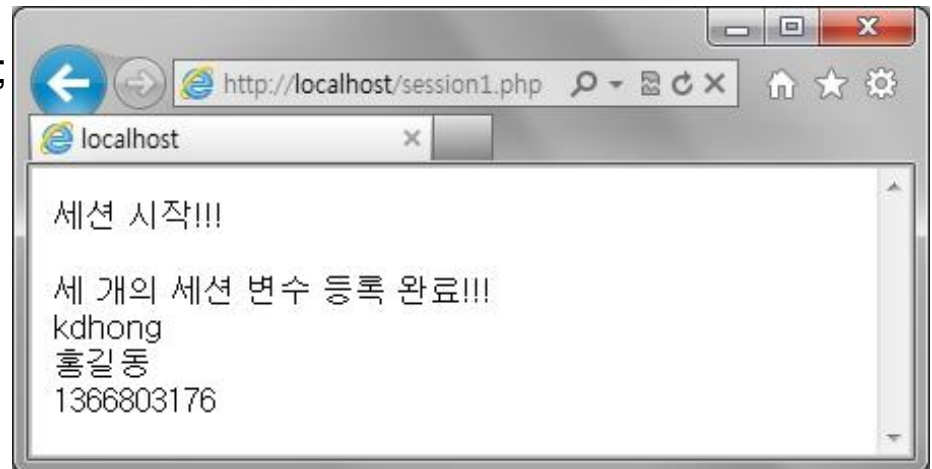
	서버	클라이언트	비교
데이터 위치	✓		
세션정보	✓	✓	서로 대응되는 정보, 같지 않음
데이터 가공	✓		

[그림] 세션의 개념과 관련 정보의 처리 장소

예제. 세션의 활성화와 등록

session1.php

```
01 <?
02 session_start();
03
04 echo "세션 시작!!!<p>";
05
06 $_SESSION['userid'] = "kdhong";
07 $_SESSION['username'] = "홍길동";
08 $_SESSION['time'] = time(); // time()은 현재 시간
09
10 echo "3개의 세션 변수 등록 완료!!!<br>";
11 echo $_SESSION['userid']."<br>";
12 echo $_SESSION['username']."<br>";
13 echo $_SESSION['time']."<br>";
14 ?>
```



HTML5 웹 스토리지

- 웹스토리지(web storage) 는 클라이언트 컴퓨터에 데이터를 저장하는 메카니즘
- 웹스토리지는 쿠키보다 안전하고 속도도 빠르다.
- 약 **5MB** 정도까지 저장 가능하다.
- 데이터는 키/값(key/value)의 쌍으로 저장



로컬 스토리지와 세션 스토리지

- **localStorage** 객체
 - 만료 날짜가 없는 데이터를 저장한다.
 - 도메인이 다르다면 서로의 로컬 스토리지에 접근할 수 없음.
- **sessionStorage** 객체
 - 각 세션(하나의 윈도우)마다 데이터가 별도로 저장
 - 해당 세션이 종료되면 데이터가 사라진다.



쿠키와 비교

기준	로컬 스토리지	세션 스토리지	쿠키
저장 공간	5-10 MB	5-10 MB	4KB
자동 해제	No	Yes	Yes
서버 측 접근성	No	No	Yes
HTTP 요청과 함께 전송	No	No	Yes
데이터 영속성	수동으로 삭제할 때까지	브라우저 탭이 닫힐 때까지	유효 기간까지

localStorage의 사용 방법

- localStorage을 사용하는 방법은 아주 간단하다. 전역 변수인 localStorage 객체에 .을 붙이고 원하는 변수 이름을 적은 후에 값을 저장하면 된다.

`localStorage.pageLoadCount = 0;`

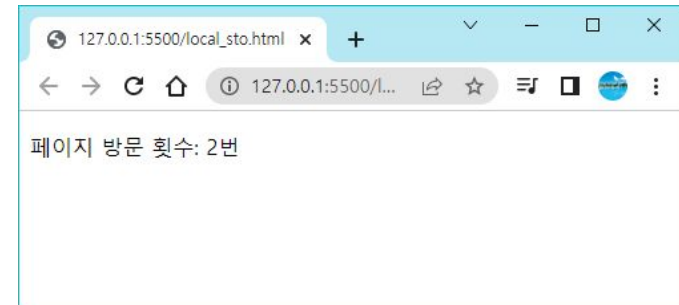
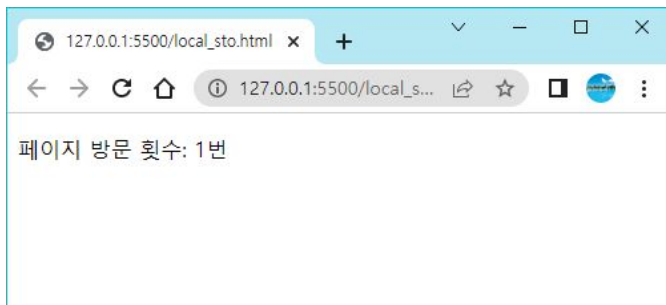
전역 변수로서 로컬 스토리지를 나타낸다.

자신이 원하는 변수 이름을 적어준다.

예제

```
<!DOCTYPE html>
<html>
<body>
  <p>    페이지 방문 횟수: <span id="count"> </span>번    </p>
  <script>
    if (!localStorage.pageLoadCount)           // (1)
      localStorage.pageLoadCount = 0;

    localStorage.pageLoadCount = parseInt(localStorage.pageLoadCount) + 1; // (2)
    document.getElementById('count').textContent = localStorage.pageLoadCount; // (3)
  </script>
</body>
</html>
```



로컬 스토리지 API

속성/메소드	설명
length	저장된 변수의 개수
setItem(key, value)	key/value를 로컬 스토리지에 저장한다.
getItem(key)	key와 연관된 값을 반환한다.
removeItem(key)	key/value를 로컬 스토리지에서 삭제한다.
clear()	모든 key/value를 삭제한다.

```
localStorage.setItem("pageLoadCount", 0);
```

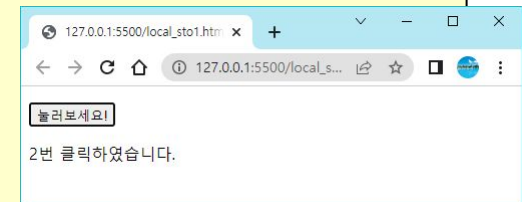
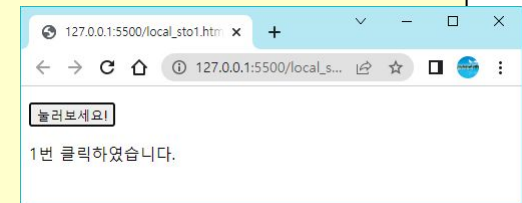
```
localStorage["pageLoadCount"] = 0;
```

```
localStorage.pageLoadCount = 0;
```

localStorage 예제

- 사용자가 버튼을 클릭한 횟수를 localStorage에 저장하는 예제를 작성한다.

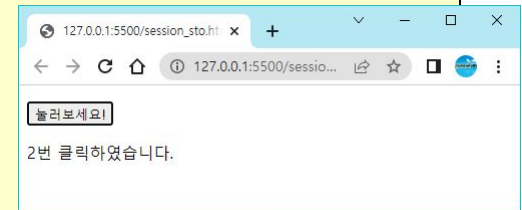
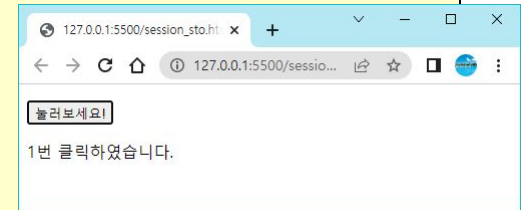
```
<!DOCTYPE html>
<html>
<body>
  <p> <button onclick="incrementCounter()" type="button">눌러보세요!</button> </p>
  <div id="target"></div>
  <script>
    function incrementCounter() {
      if (typeof(Storage) !== "undefined") {
        if (localStorage.count)
          localStorage.count++;
        else
          localStorage.count = 1;
        document.getElementById("target").innerHTML =
          localStorage.count + "번 클릭하였습니다.";
      }
      else {
        document.getElementById("target").innerHTML =
          "브라우저가 웹스토리지를 지원하지 않습니다.";
      }
    }
  </script>
</body>
</html>
```



sessionStorage 예제

...

```
<script>
function incrementCounter() {
  if (typeof(Storage) !== "undefined") {
    if (sessionStorage.count) {
      sessionStorage.count++;
    }
    else {
      sessionStorage.count = 1;
    }
    document.getElementById("target").innerHTML =
      sessionStorage.count + "번 클릭하였습니다.";
  }
  else {
    document.getElementById("target").innerHTML =
      "브라우저가 웹스토리를 지원하지 않습니다.";
  }
}
</script>
```



크롬에서의 스토리지 확인

The screenshot shows the Chrome DevTools interface with the 'Application' tab selected. The left sidebar shows the 'Storage' section expanded, with 'Session Storage' for the URL 'http://127.0.0.1:5500' highlighted. The main panel displays a table of session storage data.

127.0.0.1:5500/session_sto.html

127.0.0.1:5500/session_sto.html

눌러보세요!

1번 클릭하였습니다.

애플리케이션

- 매니페스트
- Service Workers
- 저장용량
 - 로컬 스토리지
 - http://127.0.0.1:5500
 - 세션 스토리지
 - http://127.0.0.1:5500
 - IndexedDB
 - 웹 SQL
 - 쿠키
 - 신뢰 토큰
 - 관심분야 그룹

키	값
IsThisFirstTime_Log_From_LiveServer	true
count	1

1 true

파일 API

- 파일 **API**: 웹브라우저가 사용자 컴퓨터에 있는 로컬 파일들을 읽어올 수 있도록 해주는 **API**
- PC에서 실행되는 일반적인 프로그램처럼 동작(웹 애플리케이션)
- 파일 **API**의 가장 전형적인 응용 분야는 아무래도 사용자가 파일을 선택하여서 원격 서버로 전송하는 작업
- 파일 **API**에서 사용되는 객체는 **File, FileReader**
 - **File** 객체는 로컬 파일 시스템에서 얻어지는 파일 데이터를 나타낸다.
 - **FileReader** 객체는 이벤트 처리를 통하여 파일의 데이터에 접근하는 메소드들을 제공하는 객체이다.

File 객체

- 파일 객체는 파일에 대한 정보를 가지고 있다. 그 중에서 중요한 속성은 다음과 같다.

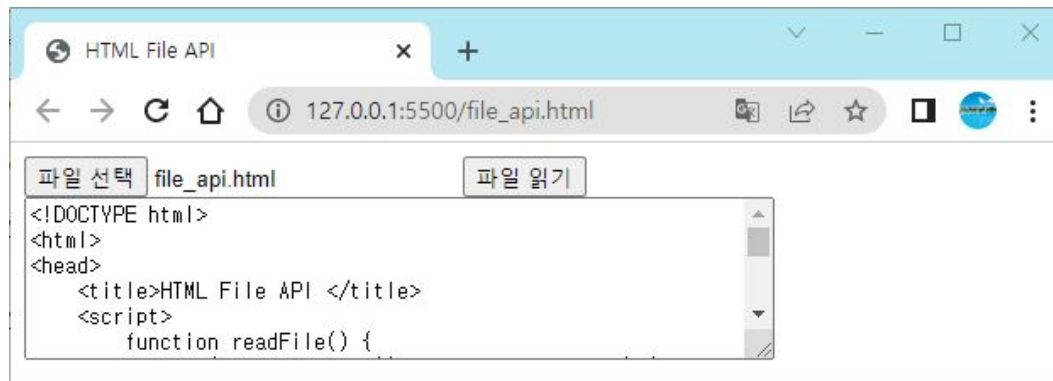
속성	설명
name	파일의 이름
size	파일의 크기(단위: 바이트)
type	파일의 타입(MIME type)
lastModifiedDate	최종 변경 날짜

예제

```
<!DOCTYPE html>
<html>
<head>
  <title>HTML File API </title>
  <script>
    function readFile() {
      if (!window.File || !window.FileReader) {
        alert('File API가 지원되지 않습니다!');
        return
      }
      let files = document.getElementById('input').files;
      if (!files.length) {
        alert('파일을 선택하십시오!');
        return;
      }
      let file = files[0];
      let reader = new FileReader();
      reader.onload = function () {
        document.getElementById('result').value = reader.result;
      };
      reader.readAsText(file, "euc-kr");
    }
  </script>
```

예제

```
</head>
<body>
  <input type="file" id="input" name="input">
  <button id="readfile" onclick="readFile()">파일 읽기</button><br />
  <textarea id="result" rows="6" cols="60"> </textarea>
</body>
</html>
```



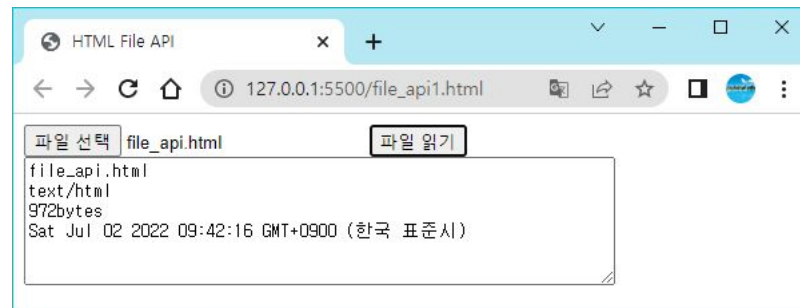
파일 정보 표시 예제

...

```
<script>
function readFile() {
    let files = document.getElementById('input').files;
    output = "";
    for (let i = 0, f; f = files[i]; i++)
    {
        output += f.name + "\n";      /* f.name - Filename */
        output += f.type + "\n";      /* f.type - File Type */
        output += f.size + "bytes\n"; /* f.size - File Size */
        output += f.lastModifiedDate + "\n"; /* f.lastModifiedDate */
    }
    document.getElementById('result').value = output;
}

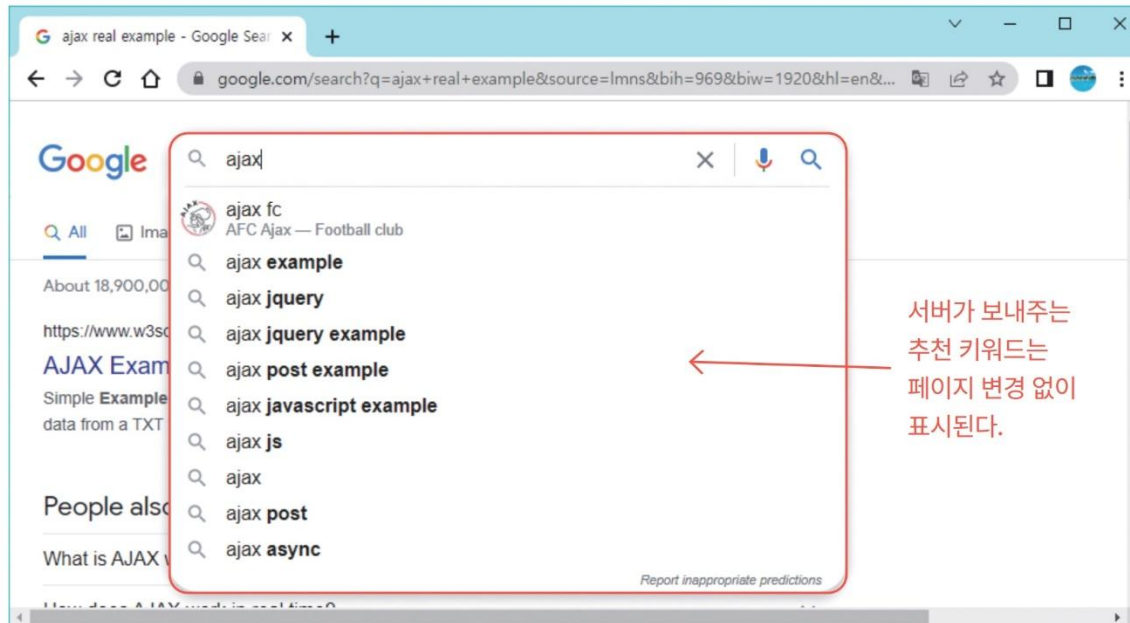
```

...



AJAX

- AJAX(Asynchronous JavaScript and XML)는 브라우저가 직접 비동기 방식으로 HTTP 통신을 하여 서버와 데이터를 교환하는 기술 중의 하나이다. AJAX는 클라이언트가 서버와 적은 양의 데이터를 교환하여 비동기적으로 HTML 페이지를 업데이트할 수 있다. 이것은 전체 페이지를 다시 적재하지 않고 웹 페이지의 일부를 업데이트할 수 있다는 것을 의미한다.

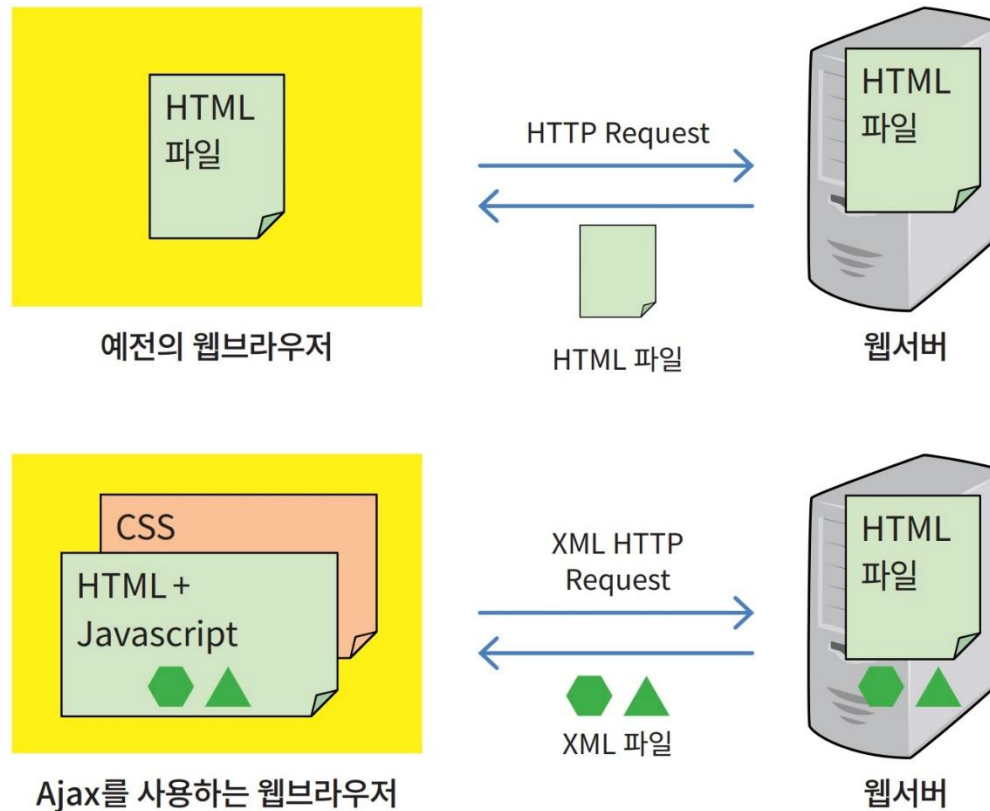


AJAX

- **AJAX**는 새로운 프로그래밍 언어가 아니라 다음과 같은 기존의 표준기술들을 현명하게 사용하는 기술이다.
 - **HTML5**(컨텐츠)
 - **CSS3** (스타일)
 - 자바스크립트, **DOM** (동적인 출력과 상호작용 담당)
 - **XML** (데이터를 전송하기 위한 형식으로 사용)
 - **XMLHttpRequest** 개체 (서버와 비동기적으로 데이터를 교환하기 위해 사용)

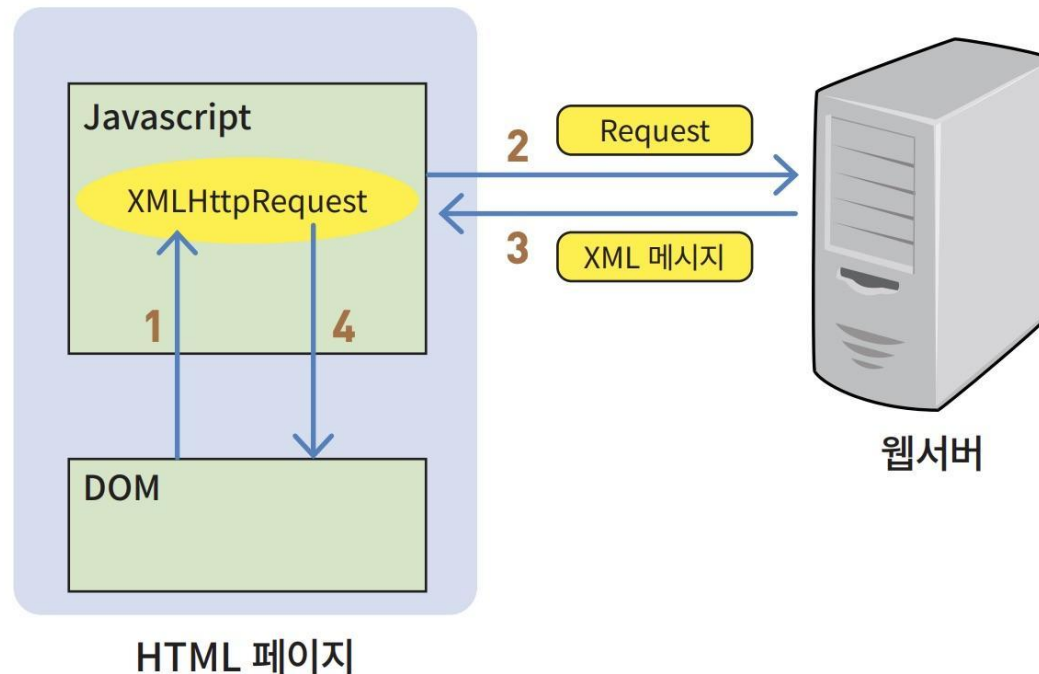
AJAX와 기존 방법의 비교

- 기존의 웹브라우저는 서버로부터 페이지 단위로만 받을 수 있었다. 반면에 AJAX를 사용하면 XML 파일 단위로 받을 수 있다.



AJAX의 동작 원리

- AJAX의 중심에 있는 기능은 페이지와 상호 작용하는 사용자를 방해하지 않으면서 비동기적으로 웹서버와 통신 할 수 있는 기능이다. XMLHttpRequest 객체를 사용하면 이것이 가능해진다.

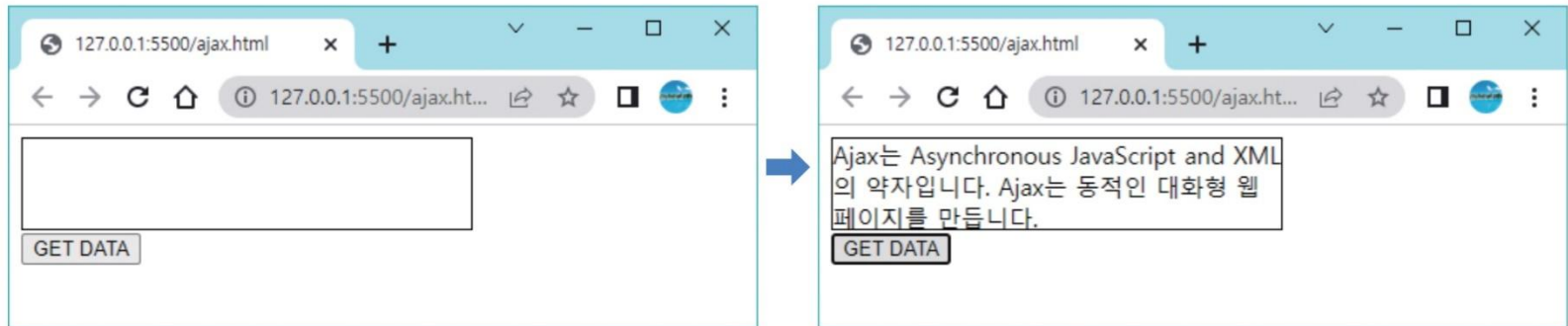


AJAX는 웹서버가 필요함

- AJAX는 필수적으로 웹서버가 필요
 - VS Code의 라이브 서버를 이용한다.
 - testfile1.txt 파일도 같은 디렉토리에 있어야 한다.
-
- HTTP Request - Response 실습은 실제 웹서버 필요
 - AWS , Dothome 같은 웹 호스팅 서버에 html 파일 업로드
 - PHP, JSP, Node.js, Flask 와 같은 백엔드를 구현해야 함.

AJAX 예제

- 사용자가 “GET DATA” 버튼을 누르면, 서버로부터 데이터 파일을 받아서 화면의 일부에 표시한다. 전체 페이지는 변경되지 않는다.



예제

testfile.txt

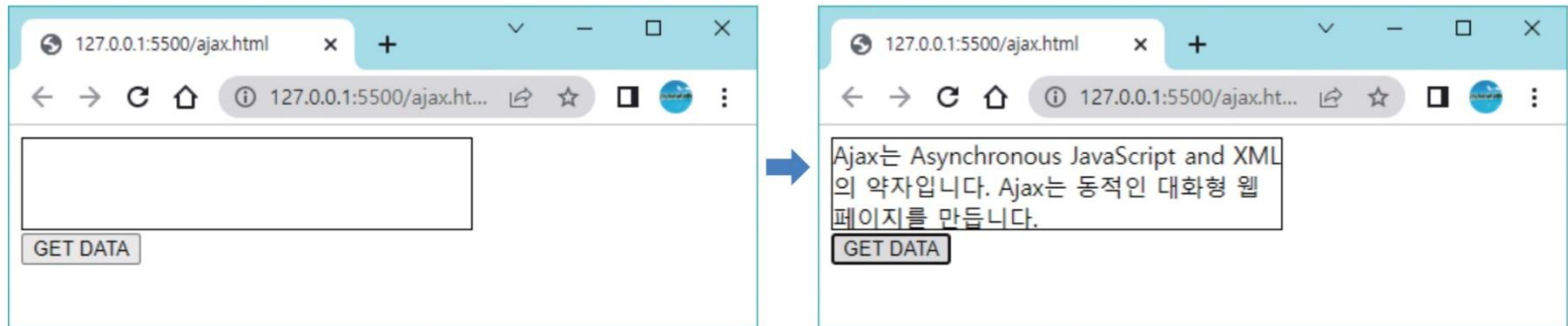
Ajax는 Asynchronous JavaScript and XML의 약자입니다.
Ajax는 동적인 대화형 웹 페이지를 만듭니다.

ajax.html

```
<!DOCTYPE html>
<html>
<head>
  <script>
    function getFromServer() {
      let req = new XMLHttpRequest();
      req.onload = function () {
        document.getElementById("target").innerHTML =
          this.responseText;
      }
      req.open("GET", "testfile1.txt", true);
      req.send();
    }
  </script>
</head>
<body>
  <div id="target" style="width: 300px; height: 60px; border: solid 1px black;">
  </div>
  <button type="button" onclick="getFromServer()">GET DATA</button>
</body>
</html>
```

실행 결과

- 사용자가 “GET DATA” 버튼을 누르면, 서버로부터 데이터 파일을 받아서 화면의 일부에 표시한다. 전체 페이지는 변경되지 않는다.



onreadystatechange 이벤트

- 서버로부터 응답이 오면 onreadystatechange 이벤트가 발생한다. onreadystatechange 이벤트는 readyState가 변경될 때마다 발생된다. ReadyState 속성은 XMLHttpRequest의 현재 상태를 가지고 있다. XMLHttpRequest 객체의 세 가지 중요한 속성은 다음과 같다.

속성	설명
onreadystatechange	readyState 상태가 변경될 때마다 자동으로 호출되는 함수를 저장한다.
readyState	XMLHttpRequest의 4가지 상태를 저장한다. 0에서 4로 순차적으로 변화된다. 0: 요청이 초기화되지 않았다. 1: 서버 연결이 이루어졌다. 2: 요청이 수신되었다. 3: 요청을 처리 중이다. 4: 요청 처리가 종료되고 응답이 준비되었다.
status	200: "OK" 404: 페이지가 발견되지 않았다.

fetch API

- **fetch API**는 서버로부터 리소스를 가져오기 위한 인터페이스를 제공한다. **AJAX**와 거의 같은 기능을 제공하면서도 **fetch API**는 더 강력하고 유연하다.
- **fetch API**는 요청(**Request**) 객체 및 응답(**Response**) 객체를 생성할 수 있다. **fetch API**는 요청 및 응답을 처리하거나, 응답을 생성해야 하는 모든 종류의 사례에 사용할 수 있다.
- **fetch()** 함수는 **window** 객체에 포함된 기능이다. 따라서 **window.fetch()**로 호출하여도 된다. **fetch()** 함수의 첫 번째 인수는 **URL**이고, 두 번째 인수는 옵션 객체이다. **fetch()** 함수는 **Promise** 타입의 객체를 반환한다.

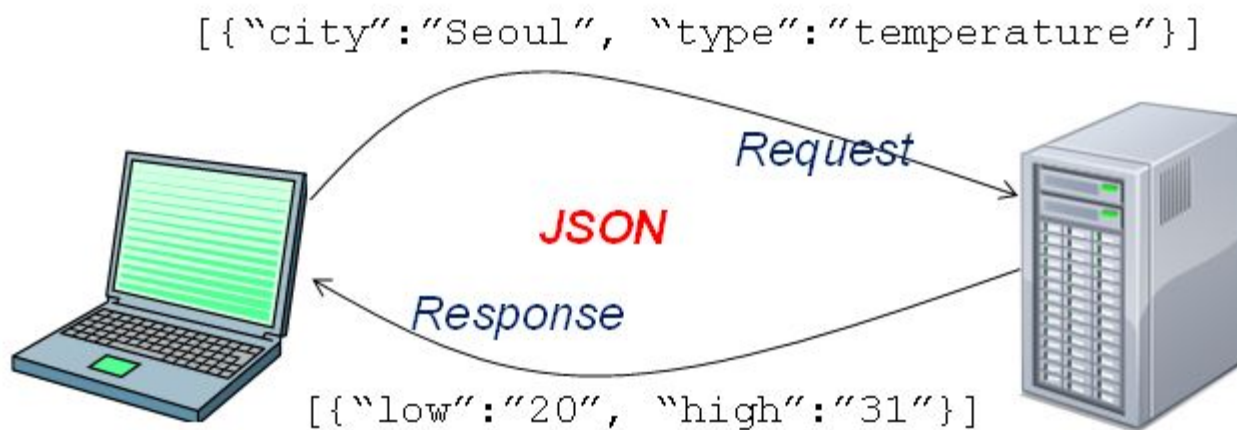
JSON

- XML 파일형식의 불편함을 개선하기 위해 등장
- JSON (JavaScript Object Notation)은 텍스트-기반의 데이터 교환 형식
- JSON은 자바 스크립트 언어에서 유래
- JSON 형식은 Douglas Crockford에 의하여 처음으로 지정되었으며, RFC 4627에 기술
- 공식적인 인터넷 미디어 타입은 `application/json`이며 파일 이름 확장자는 `.json`

JSON으로 표현된 데이터

```
{  
  "name": "HongGilDong",  
  "age": 25,  
  "address": {  
    "nation": "Korea",  
    "city": "Seoul",  
    "postalCode": "123-456"  
  },  
  "특기": ["검술", "무술"],  
  "phone": "010-123-4567"  
}
```

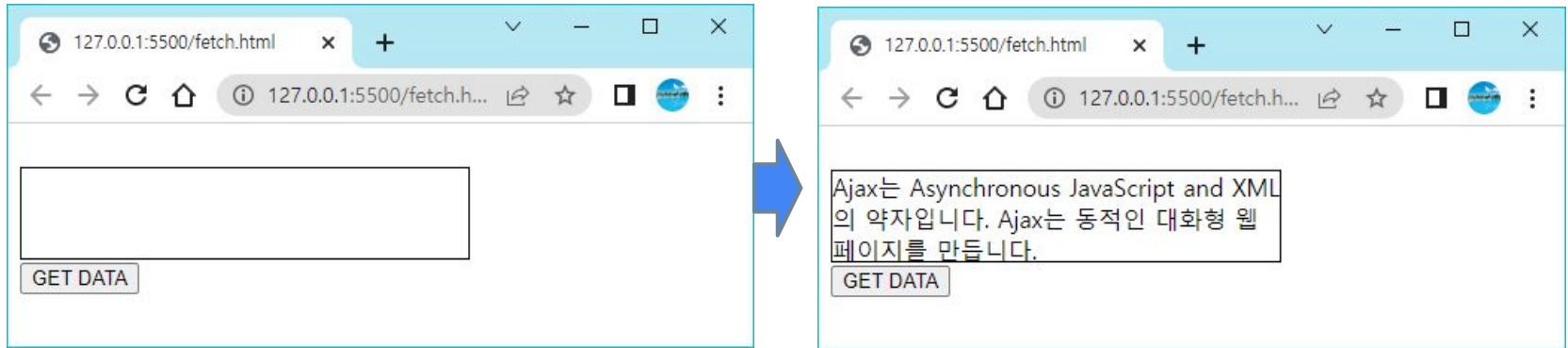

JSON의 사용



예 제

```
<!DOCTYPE html>
<html>
<head>
  <script>
    function getFromServer() {
      let file = "testfile1.txt"
      fetch(file)
        .then(response => response.text())
        .then(data => document.getElementById("target").innerHTML = data);
    }
  </script>
</head>
<body>
  <div id="target" style="width: 300px; height: 60px; border: solid 1px black;">
    </div>
  <button type="button" onclick="getFromServer()">GET DATA</button>
</body>
</html>
```

실행 결과



애플리케이션 캐시

- 애플리케이션이 사용하는 파일들을 클라이언트의 캐시(cache)에 저장
- 애플리케이션 캐시는 다음과 같은 세 가지 장점을 제공한다.
 - 오프 라인 상태일 때도 사용자는 웹 애플리케이션을 사용할 수 있다
 - 캐시된 파일은 더 빨리 로드되어서 그만큼 속도가 빨라진다.
 - 서버 부하가 감소된다.

시계 예제

clock.html

```
<!DOCTYPE HTML>
<html manifest="clock.appcache">
<head>
  <title>Clock</title>
  <script src="clock.js"></script>
  <link rel="stylesheet" href="clock.css">
</head>
<body>
  <button onclick="setClock()">시계시작 </button>
  <br />
  <output id="clock"></output>
</body>
</html>
```

clock.css

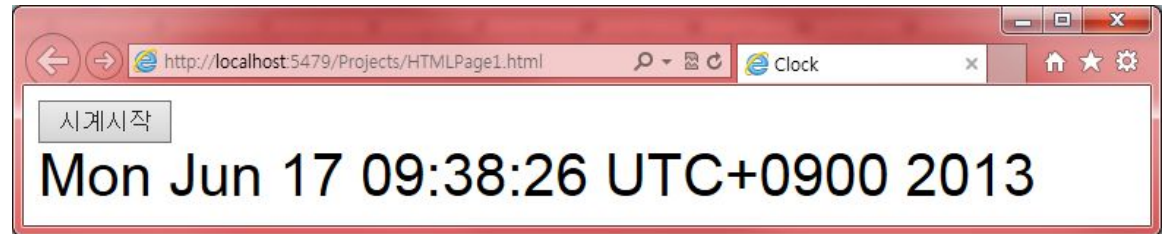
```
output {
  font: 2em sans-serif
}
```

clock.js

```
function setClock() {
  var now = new Date();
  document.getElementById('clock').innerHTML = now;
  setTimeout('setClock()', 1000);
}
```

실행 결과

인터넷 연결이
끊기면 시계는
동작할까요?



매니페스트 파일

clock.appcache

CACHE MANIFEST

clock.html

clock.css

clock.js

복잡한 매니페스트 파일

CACHE MANIFEST

2010-06-18:v2

반드시 캐시해야할 파일

CACHE:

index.html

stylesheet.css

images/logo.png

scripts/main.js

사용자가 반드시 온라인이어야 하는 리소스

NETWORK:

login.php

만약 main.jsp 가 접근될 수 없으면 static.html로 서비스한다.

다른 모든 .html파일 대신에 offline.html로 서비스한다.

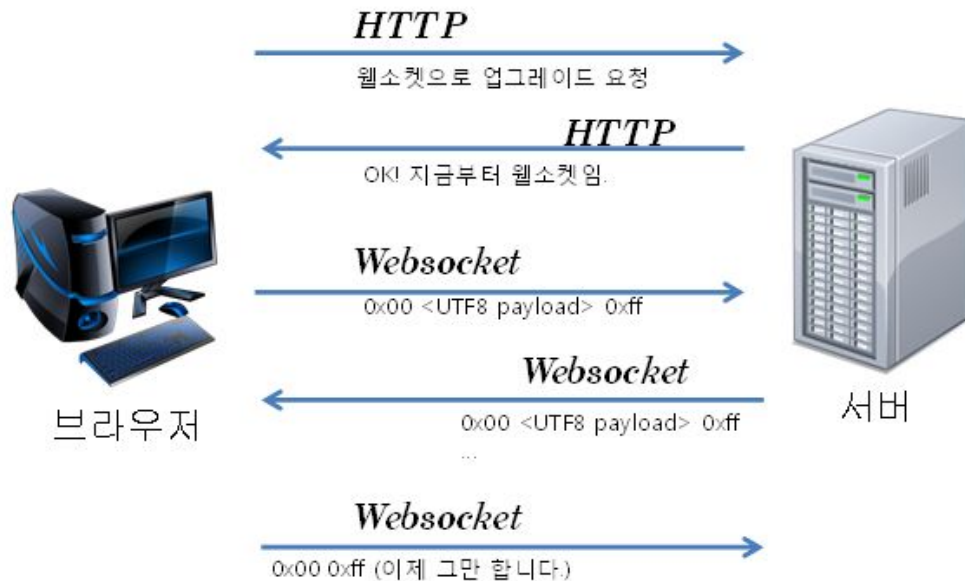
FALLBACK:

/main.jsp /static.html

*.html /offline.html

웹소켓

- **웹 소켓(Web Socket)**은 웹 애플리케이션을 위한 차세대 양방향 통신 기술
- 애플리케이션은 **HTTP**의 답답한 구속에서 벗어나서 **TCP/IP**가 제공하는 모든 기능을 사용할 수 있다.

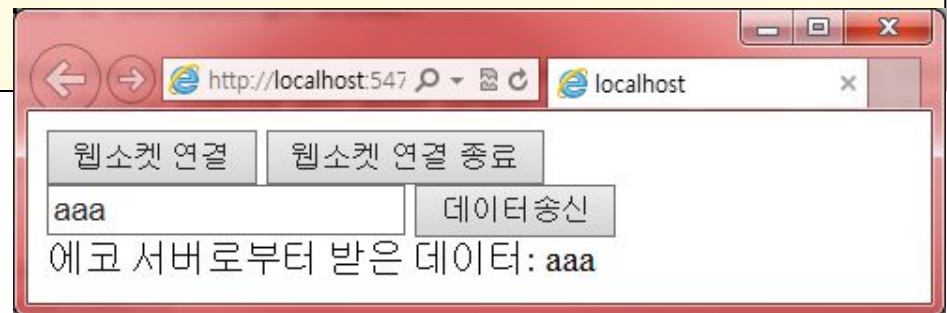


예 제

```
<!DOCTYPE HTML>
<html>
<head>
  <script>
    var ws;
    function open() {
      if ("WebSocket" in window) {
        ws = new WebSocket("ws://echo.websocket.org");
        ws.onopen = function () {
          alert("웹소켓 오픈 성공");
        };
        ws.onmessage = function (evt) {
          var msg = evt.data;
          document.getElementById("result").innerHTML = msg;
        };
        ws.onclose = function () {
          alert("웹소켓 연결 해제");
        };
      }
      else {
        alert("웹소켓이 지원되지 않음!");
      }
    }
  }
}
```

예제

```
function send() {  
    ws.send(document.getElementById("data").value);  
}  
function quit() {  
    ws.close();  
}  
</script>  
</head>  
<body>  
    <button onclick="open()">웹소켓 연결</button>  
    <button onclick="quit()">웹소켓 연결 종료</button><br />  
    <input type="text" id="data" />  
    <button onclick="send()">데이터 송신</button><br />  
    에코 서버로부터 받은 데이터:  
    <output id="result"></output>  
</body>  
</html>
```



Q & A

